

Lab_Assignment_17.2 AI Assisted Coding

B37 2303A52077

Task – 1(Student Academic Data Cleaning)

PROMPT

#Generate a Python script to clean a student dataset (students.csv) using Pandas.
#Handle missing values in Marks, Attendance, and Department
#Remove duplicate records using Student_ID
#Convert Department names to lowercase
#Encode Gender and Department into numeric values
#Display the cleaned dataset

CODE

```
ai 17.2 > 1.py > ...
1 #Generate a Python script to clean a student dataset (students.csv) using Pandas.
2 #Handle missing values in Marks, Attendance, and Department
3 #Remove duplicate records using Student_ID
4 #Convert Department names to lowercase
5 #Encode Gender and Department into numeric values
6 #Display the cleaned dataset
7 import pandas as pd
8 from sklearn.preprocessing import LabelEncoder
9
10 df = pd.read_csv(r"C:\Users\gauta\OneDrive\Documents\SEM-6\AI\docs\codes\ai 17.2\students.csv")
11
12 # Handle missing values
13 df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
14 df['Attendance'] = df['Attendance'].fillna(df['Attendance'].mean())
15 df['Department'] = df['Department'].fillna('Unknown')
16
17 # Remove duplicates
18 df.drop_duplicates(subset='Student_ID', inplace=True)
19
20 # Standardize text
21 df['Department'] = df['Department'].str.strip().str.lower()
22
23 # Fix and encode Gender
24 df['Gender'] = df['Gender'].str.strip().str.lower()
25 df['Gender'] = df['Gender'].map({'male': 1, 'female': 0})
26
27 # Encode Department
28 le = LabelEncoder()
```

OUTPUT

First dataset

2	3	103	2023-03-10	NaN	Credit
3	4	104	2023-04-05	2000.00	Credit
4	5	105	2023-05-25	-150.75	Debit
5	6	106	2023-06-30	NaN	Debit
6	7	107	2023-07-15	300.00	Credit
7	8	108	2023-08-20	-5000.00	Debit
8	9	109	2023-09-10	250.00	Credit
9	10	110	2023-10-05	-100.00	Debit

Cleaned dataset

	Student_ID	Name	Gender	Marks	Attendance	Department
0	1	Alice	NaN	85	95	0
1	2	Bob	NaN	90	80	1
2	3	Charlie	NaN	78	85	2
3	4	David	NaN	92	90	0
4	5	Eve	NaN	88	92	1

EXPLANATION

We took a student dataset and cleaned it to make it ready for analysis.

First, we filled missing values in marks, attendance, and department.

Then, we removed duplicate student records using Student_ID.

Next, we standardized text by converting department names to lowercase and removing spaces.

After that, we converted categorical data like gender and department into numbers.

Task – 2 (Financial Transaction Data Preprocessing)

PROMPT

Generate Python code to preprocess a financial transactions dataset (transactions.csv).

#Convert Transaction_Date to datetime

#Extract Month and Year

#Handle extreme values in Amount

#Normalize Amount

#Display the processed dataset

CODE

```
ai 17.2 > 2.py > ...
1 #Generate Python code to preprocess a financial transactions dataset (transactions.csv).
2 #Convert Transaction_Date to datetime
3 #Extract Month and Year
4 #Handle extreme values in Amount
5 #Normalize Amount
6 #Display the processed dataset
7 import pandas as pd
8 from sklearn.preprocessing import MinMaxScaler
9 df = pd.read_csv(r"C:\Users\gauta\OneDrive\Documents\SEM-6\AI\docs\codes\ai 17.2\transactions.csv")
10 # Convert Transaction_Date to datetime
11 df['Transaction_Date'] = pd.to_datetime(df['Transaction_Date'], errors='coerce')
12 # Extract Month and Year
13 df['Month'] = df['Transaction_Date'].dt.month
14 df['Year'] = df['Transaction_Date'].dt.year
15 # Handle extreme values in Amount using IQR
16 Q1 = df['Amount'].quantile(0.25)
17 Q3 = df['Amount'].quantile(0.75)
18 IQR = Q3 - Q1
19 lower_bound = Q1 - 1.5 * IQR
20 upper_bound = Q3 + 1.5 * IQR
21 df['Amount'] = df['Amount'].apply(lambda x: upper_bound if x > upper_bound else (lower_bound if x < lower_bound else x))
22 # Normalize Amount
23 scaler = MinMaxScaler()
24 df['Amount'] = scaler.fit_transform(df[['Amount']])
25 print(df.head())
```

OUTPUT

	Transaction_ID	User_ID	Transaction_Date	Amount	Transaction_Type	Month	Year
0	1	101	2023-01-15	0.517054	Credit	1	2023
1	2	102	2023-02-20	0.416771	Debit	2	2023
2	3	103	2023-03-10	NaN	Credit	3	2023
3	4	104	2023-04-05	1.000000	Credit	4	2023
4	5	105	2023-05-25	0.349638	Debit	5	2023

EXPLANATION

We processed the financial transaction dataset to make it ready for analysis. First, we converted the transaction date into datetime format. Then, we extracted useful features like month and year from the date. Next, we identified and handled extreme values (very high or low amounts). After that, we normalized the amount values so all values are in the same scale. Finally, we obtained a clean and structured dataset with new features

Task 3 – (Environmental Sensor Data Preparation)

PROMPT

#Generate Python code to preprocess an environmental sensor dataset (sensor_data.csv).
#Handle missing values in Temperature, Humidity, and Air_Quality_Index
#Scale numerical features using standard scaling
#Encode Sensor_Status into numeric values
#Split the dataset into training and testing sets
#Display the processed data

CODE

```
#Generate Python code to preprocess an environmental sensor dataset (sensor_data.csv).
#Handle missing values in Temperature, Humidity, and Air_Quality_Index
#Scale numerical features using standard scaling
#Encode Sensor_Status into numeric values
#Split the dataset into training and testing sets
#Display the processed data
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv(r"C:\Users\gauta\OneDrive\Documents\SEM-6\AI\docs\codes\ai_17.2\sensor_data.csv")
# Handle missing values
data['Temperature'] = data['Temperature'].fillna(data['Temperature'].mean())
data['Humidity'] = data['Humidity'].fillna(data['Humidity'].mean())
data['Air_Quality_Index'] = data['Air_Quality_Index'].fillna(data['Air_Quality_Index'].mean())
# Fix and encode Sensor_Status
data['Sensor_Status'] = data['Sensor_Status'].str.strip().str.lower()
data['Sensor_Status'] = data['Sensor_Status'].map({'active': 1, 'inactive': 0})
# Scale numerical features
scaler = StandardScaler()
features = ['Temperature', 'Humidity', 'Air_Quality_Index']
data[features] = scaler.fit_transform(data[features])
# Split dataset
X = data.drop('Sensor_Status', axis=1)
y = data['Sensor_Status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Output
print("Training Data:\n", X_train.head())
print("\nTesting Data:\n", X_test.head())
print("\nTrain Labels:\n", y_train.head())
print("\nTest Labels:\n", y_test.head())
```

OUTPUT

```
Training Data:
  Sensor_ID Temperature Humidity Air_Quality_Index
5  Sensor_6    1.355529 -1.521354        -0.737801
3  Sensor_4    -1.633854 -1.163319        -1.542676
10 Sensor_11     0.859673  0.344284        -0.782517
14 Sensor_15    -1.839036 -1.041739         0.029810
4  Sensor_5     1.059156  0.582414         0.335364

Testing Data:
  Sensor_ID Temperature Humidity Air_Quality_Index
12 Sensor_13    1.221591  0.819707         1.378720
7  Sensor_8    -0.699138  1.172710        -0.171408
6  Sensor_7     0.056045  0.799583         1.319099

Train Labels:
5      0
3      0
10     0
14     1
4      0
Name: Sensor_Status, dtype: int64

Test Labels:
12     1
7      1
6      0
Name: Sensor_Status, dtype: int64
```

EXPLANATION

We cleaned and prepared the environmental sensor dataset for analysis.

First, we handled missing values in temperature, humidity, and air quality index by filling them with the mean. Then, we scaled numerical features using standard scaling so all values are on the same range. Next, we encoded sensor status into numeric values. Finally, we split the dataset into training and testing sets for model building.

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

PROMPT

```
#Generate Python code to clean and preprocess a traffic dataset
(traffic_data.csv).#Standardize Road_Name and Location text#Fill missing
Traffic_Density values #Remove duplicate records #Normalize Speed and
Vehicle_Count #Display summary before and after cleaning
```

CODE

```
ai 17.2 > 4.py > ...
1 #Generate Python code to clean and preprocess a traffic dataset (traffic_data.csv).
2 #Standardize Road_Name and Location text
3 #Fill missing Traffic_Density values
4 #Remove duplicate records
5 #Normalize Speed and Vehicle_Count
6 #Display summary before and after cleaning
7 import pandas as pd
8 from sklearn.preprocessing import MinMaxScaler
9 # Load dataset
10 data = pd.read_csv(r"C:\Users\gauta\OneDrive\Documents\SEM-6\AI\docs\codes\ai 17.2\traffic_data.csv")
11 # Display summary before cleaning
12 print("Summary before cleaning:\n", data.describe())
13 # Standardize text
14 data['Road_Name'] = data['Road_Name'].str.strip().str.title()
15 data['Location'] = data['Location'].str.strip().str.title()
16 # Fill missing values
17 data['Traffic_Density'] = data['Traffic_Density'].fillna(data['Traffic_Density'].mean())
18 # Remove duplicates
19 data = data.drop_duplicates()
20 # Normalize numerical features
21 scaler = MinMaxScaler()
22 data[['Speed', 'Vehicle_Count']] = scaler.fit_transform(data[['Speed', 'Vehicle_Count']])
23 # Display summary after cleaning
24 print("\nSummary after cleaning:\n", data.describe())
25 # Display cleaned data
26 print("\nCleaned Data:\n", data.head())
27
```

OUTPUT

```
Summary before cleaning:
count    Sensor_ID  Traffic_Density    Speed  Vehicle_Count
count    12.000000         8.000000  12.000000         12.00000
mean      6.500000       193.750000  22.250000        143.75000
std       3.605551       97.970477   7.225271         81.27185
min       1.000000      100.000000  10.000000         50.00000
25%       3.750000      100.000000  17.250000         68.75000
50%       6.500000      175.000000  23.500000        137.50000
75%       9.250000      262.500000  28.500000        206.25000
max       12.000000      350.000000  30.000000        275.00000

Summary after cleaning:
count    Sensor_ID  Traffic_Density    Speed  Vehicle_Count
count    12.000000        12.000000  12.000000        12.00000
mean      6.500000       193.750000   0.612500         0.416667
std       3.605551       78.153404   0.361264         0.361208
min       1.000000      100.000000   0.000000         0.000000
25%       3.750000      137.500000   0.362500         0.083333
50%       6.500000      193.750000   0.675000         0.388889
75%       9.250000      212.500000   0.925000         0.694444
max       12.000000      350.000000   1.000000         1.000000
```

EXPLANATION

We cleaned the traffic dataset to improve its quality. First, we standardized road and location names by converting them to lowercase and removing extra spaces. Then, we filled missing traffic density values using the mean. Next, we removed duplicate records to avoid repeated data. After that, we normalized speed and vehicle count so all values are on the same scale. Finally, we compared the dataset before and after cleaning to see the improvement.

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

PROMPT

#Generate Python code to clean social media text data.
#Remove URLs, mentions, hashtags, and special characters
#Convert text to lowercase
#Tokenize the text
#Remove stop words
#Create features like word count and sentiment score
#Display old and processed dataset

CODE

```
ai 17.2 > 5.py > clean_text
1  #Generate Python code to clean social media text data.
2  #Remove URLs, mentions, hashtags, and special characters
3  #Convert text to lowercase
4  #Tokenize the text
5  #Remove stop words
6  #Create features like word count and sentiment score
7  #Display old and processed dataset
8  import pandas as pd
9  import re
10
11 # Load dataset
12 data = pd.read_csv(r"C:\Users\gauta\OneDrive\Documents\SEM-6\AI\docs\codes\ai 17.2\social_media_posts
13
14 # Display original dataset
15 print("Original Dataset:\n", data.head())
16 data.drop_duplicates(inplace=True)
17 data['Text'] = data['Text'].fillna("") # handle empty values
18 def clean_text(text):
19     text = str(text)
20     text = re.sub(r'http\S+|www\S+|https\S+', '', text)
21     text = re.sub(r'@\w+|#\w+', '', text)
22     text = re.sub(r'^a-zA-Z\s', '', text)
23     text = text.lower()
24     text = re.sub(r'\s+', ' ', text).strip()
25     return text
26
27 data['Cleaned_Text'] = data['Text'].apply(clean_text)
28
29 stop_words = ['is', 'the', 'and', 'a', 'an', 'in', 'on', 'at', 'to', 'for', 'of', 'with', 'this', 'that', 'it']
30
31 def process_text(text):
32     words = text.split()
33     words = [w for w in words if w not in stop_words]
```

```

35
36 data['Tokens'] = data['Cleaned_Text'].apply(process_text)
37 # Word count
38 data['Word_Count'] = data['Tokens'].apply(len)
39
40 # Simple sentiment (rule-based)
41 positive_words = ['good', 'great', 'happy', 'love', 'excellent', 'awesome']
42 negative_words = ['bad', 'sad', 'hate', 'worst', 'poor', 'angry']
43
44 def get_sentiment(words):
45     score = 0
46     for w in words:
47         if w in positive_words:
48             score += 1
49         elif w in negative_words:
50             score -= 1
51     return score
52
53 data['Sentiment'] = data['Tokens'].apply(get_sentiment)
54 print("\nProcessed Dataset:\n", data.head())

```

OUTPUT

```

Original Dataset:
  Post_ID      Text
0      1  Hello, world!
1      2  Check out this amazing URL: https://example.com
2      3      This is a duplicate post.
3      4  Another duplicate post.
4      5  Some empty text here.

Processed Dataset:
  Post_ID      Text  ... Word_Count Sentiment
0      1  Hello, world!  ...         2         0
1      2  Check out this amazing URL: https://example.com  ...         4         0
2      3      This is a duplicate post.  ...         2         0
3      4  Another duplicate post.  ...         3         0
4      5  Some empty text here.  ...         4         0

[5 rows x 6 columns]

```

EXPLANATION

We cleaned and processed social media text data to prepare it for sentiment analysis. First, we removed duplicate posts and handled empty text values. Then, we cleaned the text by removing URLs, special characters, and extra spaces. Next, we converted text to lowercase and split it into words. After that, we removed common stopwords to keep only meaningful words. Finally, we extracted features like word count and a simple sentiment score. This makes the text data clean and suitable for analysis.